NTP Sınav Hazırlık Çalışma Kağıdı

Nesne Tabanlı Programlama (C#) · 10. sınıflar · Oct 18, 2026

Bu kağıttaki 11 konunun hepsi sınavda karşına çıkacak; her konuyu okuyup örnek kodları kendi defterinde adım adım izlersen sınava hazır olursun.

Nasıl çalışmalısın:

1. Her bölümde önce "Bilmen gerekenler" kısmını oku.
2. Örnek kodun çıktısını önce kendin tahmin et, sonra açıklamaya bak.
3. "Merak edenlere" kutuları sınavda sorulmaz; konuyu sevdirmek için var.
4. En sondaki "Kendini dene" sorularını kâğıda yazarak çöz, cevapları en son kontrol et.

Kod dili C#. Sınavda çıktıları satır sırasıyla yazman istenecek; bu yüzden kodu satır satır izleme alışkanlığı kazan.

1. Algoritma ve veri türleri

Algoritma, bir problemi çözmek için izlenen, sırası belli ve sonlu adımlar dizisidir. Sıra önemlidir: bir bilgiyi kullanmadan önce almak zorundasın.

Örnek: Bir öğrenci kartını ekranda göstermek

1. Başla
2. Öğrencinin adını al
3. Numarasını al
4. Kartı göster
5. Bitir

"Kartı göster" adımı en başa yazılsaydı kartta gösterilecek bilgi olmazdı. Algoritmayı "Başla" ve "Bitir" ile çerçevelemek iyi bir alışkanlıktır.

Veri türleri, bir değişkenin ne tür değer tutacağını söyler.

Tür	Tuttuğu değer	Örnek
string	Metin, çift tırnak içinde	string sehir = "Bursa";
int	Tam sayı	int yas = 16;
double	Ondalıklı sayı	double boy = 1.72;
bool	Yalnız true ya da false	bool geldi = true;
char	Tek karakter, tek tırnak içinde	char harf = 'A';

Dikkat: "25" (tırnaklı) bir metindir, 25 (tırnaksız) bir sayıdır. bool değerler tırnaksız ve küçük harfle yazılır.

> **Merak edenlere:** "Algoritma" kelimesi, 9. yüzyılda yaşamış matematikçi el-Harezmî'nin adının Latince yazılışından gelir. Bir int, yaklaşık -2,1 milyar ile +2,1 milyar arasındaki sayıları tutabilir; daha büyük sayılar için long kullanılır.

2. Sınıf ve nesne

Sınıf (class) bir kalıp ya da tarif, **nesne (object)** o kalıptan üretilmiş somut bir örnektir. Sınıf "bir öğrencinin adı ve notu olur" der; nesne "Ayşe, notu 85" gibi gerçek bir öğrencidir.

```
class Ogrenci
{
    public string Ad;
    public int Not;
}

Ogrenci o1 = new Ogrenci();   // birinci nesne
Ogrenci o2 = new Ogrenci();   // ikinci nesne
o1.Ad = "Ayse"; o1.Not = 85;
o2.Ad = "Mert"; o2.Not = 70;
```

Aynı sınıftan üretilen nesnelerin alanları aynıdır (ikisinin de Ad ve Not alanı var), ama alanlardaki **değerler farklı olabilir**. Her nesne kendi değerlerini taşır.

- `new` anahtar kelimesi bellekte yeni bir nesne oluşturur.
- Sınıf bir kez yazılır, ondan istediğin kadar nesne üretebilirsin.

> **Merak edenlere:** C# dili 2000 yılında Microsoft'ta Anders Hejlsberg liderliğindeki bir ekip tarafından geliştirildi. Adındaki "#" işareti, müzikteki diyez işaretinden esinlenmiştir: notayı yarım ton yükseltir, yani "C'nin bir adım ötesi" anlamına gelir.

3. Operatörler ve işlem önceliği

C# matematikteki kuralı uygular: **önce parantez, sonra çarpma/bölme, en son toplama/çıkarma**. Aynı öncelikteki işlemler soldan sağa yapılır.

```
Console.WriteLine(10 + 4 * 2);     // 4*2=8, 10+8 -> 18
Console.WriteLine((10 + 4) * 2);   // 10+4=14, 14*2 -> 28
Console.WriteLine(12 - 6 / 3);     // 6/3=2, 12-2 -> 10
```

Operatör	Anlamı	Örnek	Sonuç
*	Çarpma	6 * 3	18
/	Bölme (int ile küsurat atılır)	7 / 2	3
%	Bölümden kalan	7 % 2	1
+	Toplama	6 + 3	9
-	Çıkarma	6 - 3	3

Atama ile eşitlik farklıdır: `=` değer atar, `==` iki değerin eşit olup olmadığını sorar. `x = x + 5;` ile `x += 5;` aynı işi yapar.

> **Merak edenlere:** "7 / 2" C#'ta 3 verir ama "7.0 / 2" 3.5 verir. İki tam sayıyı böldüğünde sonuç da tam sayı olur. Bu, programcıların en sık yaptığı hatalardan biridir.

4. Form kontrolleri ve olaylar

Pencereli (form) uygulamalarda kullanıcıyla iletişimi **kontroller** kurar; en temel üçlü metin almak, işlemi başlatmak ve sonucu göstermek içindir.

Kontrol	Görevi	Sık kullanılan özelliği
TextBox	Kullanıcıdan metin alır	Text
Button	Bir işlemi başlatır	Text (üzerindeki yazı)

Kontrol	Görevi	Sık kullanılan özelliği
Label	Ekranda yazı/sonuç gösterir	Text
CheckBox	İşaretli/işaretsiz seçim	Checked
ListBox	Birden fazla öğeyi listeler	Items

Olay (event), programda bir şeyin olmasıdır: düğmeye tıklanması, metnin değişmesi, formun açılması gibi. Düğmeye basılınca **Click** olayı tetiklenir. Bu olaya bağlı metot (olay metodu) **yalnızca olay gerçekleştiğinde**, yani kullanıcı düğmeye tıkladığı anda çalışır; program açılınca kendiliğinden çalışmaz.

```
private void btnSelamla_Click(object sender, EventArgs e)
{
    lblSonuc.Text = "Merhaba " + txtAd.Text;
}
```

Bu örnekte kullanıcı TextBox'a adını yazar, Button'a basar, sonuç Label'da görünür.

> **Merak edenlere:** Kontrollere ad verirken ön ek kullanmak eski bir gelenektir: btn (Button), txt (TextBox), lbl (Label). Böylece kodda bir adın hangi kontrole ait olduğunu hemen anlarsın.

5. Güvenli sayı girişi: TryParse

`Console.ReadLine()` kullanıcının yazdığını her zaman **metin (string)** olarak getirir. Bu metni sayıya çevirmenin güvenli yolu `int.TryParse`'dır.

- Çeviri başarılıysa **true** döner ve sayı `out` ile verilen değişkene yazılır.
- Çeviri başarısızsa **false** döner; program çökmez.

```
int yas;
if (int.TryParse(Console.ReadLine(), out yas))
    Console.WriteLine("Yasin: " + yas);
else
    Console.WriteLine("Lutfen sayi gir.");
```

Kullanıcının girdiği	TryParse sonucu	Ekranda görünen
16	true	Yasin: 16
on altı	false	Lutfen sayi gir.

Kullanıcının girdiği	TryParse sonucu	Ekranda görünen
3.5	false (tam sayı değil)	Lutfen sayi gir.
(hiçbir şey yazmadan Enter)	false (boş metin)	Lutfen sayi gir.

`int.Parse` ise geçersiz bir metinde hata verip programı durdurur. Bu yüzden kullanıcı girişinde TryParse tercih edilir.

> **Merak edenlere:** TryParse başarısız olduğunda `out` değişkenine 0 yazar. Yani yukarıdaki örnekte "on altı" girilince yas değişkeni 0 olur, ama bu değer ekrana yazılmaz.

6. Koşullar, sınır değerleri ve hata türleri

`if` bir koşulu kontrol eder; koşul true ise ilk blok, false ise `else` bloğu çalışır. Karşılaştırma operatörlerini karıştırmamak çok önemlidir.

Operatör	Anlamı	Sözle ifadesi
>	büyüktür	"...den fazla"
>=	büyük veya eşittir	"en az ..."
<	küçüktür	"...den az"
<=	küçük veya eşittir	"en fazla ..."
==	eşittir	"tam olarak ..."
!=	eşit değildir	"...den farklı"

Sınır değerini mutlaka dene. Kural "not en az 45 ise geçer" olsun:

```
if (notu > 45)
    Console.WriteLine("Gecti");
else
    Console.WriteLine("Kaldi");
```

not	Kodun çıktısı	Kurala göre doğru olan
44	Kaldi	Kaldi
45	Kaldi	Gecti (hata burada!)

not	Kodun çıktısı	Kurala göre doğru olan
46	Gecti	Gecti

"En az 45" 45'i de içerir; doğru koşul `notu >= 45` olmalıdır. Sınırın bir altı, kendisi ve bir üstü denenince hata hemen ortaya çıkar.

Hata türleri:

- **Derleme (sözdizimi) hatası:** Kod hiç çalışmaz. Örnek: noktalı virgül unutmak, bir değişkeni tanımlamadan kullanmak.
- **Mantık hatası:** Kod çalışır ama yanlış sonuç verir. Yukarıdaki `>` yerine `>=` yazmamak tam olarak budur.
- **Çalışma zamanı hatası:** Kod derlenir ama çalışırken çöker. Örnek: sıfıra bölme, `int.Parse("abc")`.

> **Merak edenlere:** Program hatalarına "bug" (böcek) denmesinin ünlü bir hikâyesi var: 1947'de Grace Hopper'ın ekibi, Harvard Mark II bilgisayarının bir rölesine sıkışmış gerçek bir güve buldu ve onu kayıt defterine yapıştırdı.

7. Döngüler: for ve while

Döngü, bir kod bloğunu koşul sağlandığı sürece tekrar tekrar çalıştırır. Kodu izlerken her turda değişkenlerin değerini bir tabloya yazmak en güvenli yöntemdir.

for döngüsü — tekrar sayısı belliyse kullanılır. Üç parçası vardır: başlangıç; koşul; artış.

```
int para = 100;
for (int gun = 1; gun <= 4; gun++)
{
    para = para - 15;
    Console.WriteLine(para);
}
```

gun	Koşul (gun <= 4)	para	Ekrana yazılan
1	true	85	85
2	true	70	70
3	true	55	55
4	true	40	40

gun	Koşul (gun <= 4)	para	Ekrana yazılan
5	false	—	döngü biter

Gövde 4 kez çalıştı. Koşul 5 kez kontrol edildi (sonuncusu false).

while döngüsü — "koşul doğru olduğu sürece" çalışır. Koşuldaki değişken gövdede **hiç değişmezse** koşul hep true kalır ve **sonsuz döngü** oluşur.

```
int su = 0;
while (su < 9)
{
    Console.WriteLine(su);
    // su hiç artmıyor -> ekrana sonsuza kadar 0 yazılır!
}
```

Düzeltmek için gövdeye `su = su + 3;` (ya da `su += 3;`) eklenir. WriteLine satırından sonra eklenirse ekrana 0, 3, 6 yazılır (öncesine eklenirse 3, 6, 9); döngü 3 kez çalışır ve su 9 olduğunda koşul false olur.

> **Merak edenlere:** Sonsuz döngüye giren bir konsol programını Ctrl + C tuşlarıyla durdurabilirsin. Bazı programlar, örneğin oyunlar, bilerek sonsuz döngü kullanır: oyun kapatılana kadar ekranı saniyede onlarca kez yeniden çizer.

8. Metotlar, parametre ve return

Metot, bir işi yapan ve adıyla çağrılan kod bloğudur. Aynı işi tekrar tekrar yazmak yerine bir kez yazıp istediğin yerde çağırırsın.

```
static int KareAl(int sayi)
{
    return sayi * sayi;
}
// Main icinde:
int sonuc = KareAl(7);
Console.WriteLine(sonuc);   // 49
```

Parça	Görevi
int (baştaki)	Metodun döndüreceği değerin türü
KareAl	Metodun adı

Parça	Görevi
int sayi	**Parametre**: metoda dışarıdan gelen değeri tutar (burada 7)
return	Hesaplanan sonucu metodu çağıran yere **geri gönderir** ve metodu bitirir
KareAl(7)	**Çağrı**: 7 değeri parametreye aktarılır

void metotlar değer döndürmez; yalnızca bir iş yapar (örneğin ekrana yazar ya da bir alanı değiştirir). Bu metotlarda return ile değer gönderilmez.

> **Merak edenlere:** `Main` da bir metottur ve programın başlangıç noktasıdır. Bir program çalıştığında bilgisayar ilk olarak Main metodunu arar. `static` kelimesi, metodu kullanmak için nesne oluşturmaya gerek olmadığını söyler.

9. Nesnelerin ayrı durumu ve kapsülleme

Her nesnenin kendi alanları vardır. Bir nesnede yapılan değişiklik, aynı sınıftan üretilmiş başka bir nesneyi etkilemez.

```
class Hesap { public int Bakiye; }

Hesap x = new Hesap();
Hesap y = new Hesap();
x.Bakiye = 200;
y.Bakiye = 300;
x.Bakiye = x.Bakiye - 50;
Console.WriteLine(x.Bakiye);   // 150
Console.WriteLine(y.Bakiye);   // 300
```

x ve y iki ayrı `new` ile oluşturulduğu için bellekte iki ayrı nesnedir. x'in bakiyesi azalınca y'ninki aynı kalır.

Kapsülleme (encapsulation), bir nesnenin verisini dışarıdan doğrudan değiştirilmeye karşı korumaktır.

- `private` alan: Yalnızca sınıfın kendi içinden erişilebilir.
- `public` özellik (property): Dışarıya açılan kapıdır. `get` okumayı, `set` yazmayı sağlar.

```
class Hesap
{
    private int bakiye = 200;
    public int Bakiye
    {
        get { return bakiye; }   // set yok!
    }
}
```

Dışarıda `Hesap h = new Hesap();` ile bir nesne oluşturulduğunu düşün:

Dışarıdaki kod	Sonuç
`Console.WriteLine(h.Bakiye);`	Çalışır, 200 yazar (get var)
`h.Bakiye = 5000;`	**Derleme hatası**: özellik salt okunurdur (set yok)
`h.bakiye = 5000;`	**Derleme hatası**: alan private

Böylece bakiye yalnızca sınıfın kendi metotlarıyla, sınıfın kurallarına uyarak değişir. Dışarıdaki kod kuralı çiğneyen bir değer (örneğin eksi bakiye) yazamaz.

> **Merak edenlere:** Nesne tabanlı programlamanın dört temel taşı vardır: kapsülleme, kalıtım (inheritance), çok biçimlilik (polymorphism) ve soyutlama (abstraction). Bu yılın konusu kapsüllemedir; diğerleri ileride gelecek.

10. Uygulama türleri ve ortak sınıf

Aynı C# kodu çok farklı ekranlarda çalışabilir. Bir uygulama iki katmana ayrılır: **kural (iş mantığı)** ve **arayüz (ekran)**.

Uygulama türü	Nerede çalışır
Konsol	Siyah komut penceresinde, yazıyla
WinForms	Windows masaüstünde, sürükle-bırak formlarla
WPF	Windows masaüstünde, daha esnek ve modern görünümle
.NET MAUI	Tek kodla Android, iOS, Windows ve macOS'ta
Blazor	Web tarayıcısında

Kural her sürümde ayrı yazılırsa ne olur? Diyelim ki bir quiz uygulamasının geçme notu 50'den 60'a çıkacak. Kural beş yerde ayrı ayrı yazılmışsa beş yeri de değiştirmen gerekir. Birini unutursan sürümler farklı sonuç verir: telefonda geçen öğrenci web'de kalır. Bu hem zaman kaybı hem de tutarsızlık demektir.

Çözüm: ortak sınıf. Kuralı tek bir sınıfa (örneğin `QuizKurali`) yazıp tüm sürümlerin onu kullanmasını sağla.

Katman	Sorumluluğu
Ortak sınıf (QuizKurali)	Kuralları tutar ve uygular: geçme notu, puan hesabı, cevap kontrolü
Arayüz (her sürüm)	Kullanıcıdan girişi alır ve sonucu ekranda gösterir

Kural değişince yalnızca ortak sınıf güncellenir, beş sürüm birden doğru çalışır.

> **Merak edenlere:** Programcılar bu fikre "DRY" der: Don't Repeat Yourself, yani "Kendini tekrar etme". Aynı bilgiyi tek bir yerde tutmak, büyük yazılım şirketlerinin en çok önem verdiği kurallardan biridir.

11. Kural yazan metotlar (uygulama sorularına hazırlık)

Sınavdaki en yüksek puanlı sorular, bir sınıfın içine kuralı uygulayan bir metot yazmanı ister. İki kalıbı iyi öğren: **üst sınırlı artırma** ve **koşullu azaltma**.

Kalıp 1 — Üst sınırlı artırma (void): Bir telefonun pili 0–100 arasındadır. `Sarj()` pili 25 artırsın; 100'ü aşarsa 100 olsun.

```
class Telefon
{
    private int pil = 40;
    public void Sarj()
    {
        pil = pil + 25;
        if (pil > 100)
            pil = 100;
    }
}
```

Başlangıç pil	Sarj() sonrası	Neden
40	65	Sınırı aşmadı

Başlangıç pil	Sarj() sonrası	Neden
80	100	105 oldu, 100'e indirildi
100	100	125 oldu, 100'e indirildi

void metot değer döndürmez; işi alanı değiştirmektir. Önce artır, sonra sınırı kontrol et.

Kalıp 2 — Koşullu azaltma (bool): `Oyna()` metodu pil 30'dan azsa pili değiştirmeden false dönsün; aksi hâlde pil 30 azalsın ve true dönsün.

```
public bool Oyna()
{
    if (pil < 30)
        return false;     // pil degismez
    pil = pil - 30;
    return true;
}
```

Başlangıç pil	Son pil	Dönüş değeri
29	29	false
30	0	true
75	45	true

Dikkat edilecekler:

- `return` çalıştığı anda metot biter; alttaki satırlar çalışmaz. Bu yüzden false durumunda pil azalmaz.
- bool döndüren bir metotta her yol bir `return` ile bitmelidir; yoksa derleme hatası alırsın.
- Sınırı (burada 30) mutlaka dene: "30'dan az" ifadesi 30'u kapsamaz, yani `<` kullanılır.
- Uyarıyı (örneğin "Pil yetersiz") metodun içinde değil, false dönüşünü gören dışarıdaki kod gösterir.

> **Merak edenlere:** Gerçek telefonlar pili hiçbir zaman tam %0'a indirmez; ekranda %0 yazdığında bile pilin içinde, onu korumak için küçük bir yedek enerji kalır. Bu da aslında bir "sınır kuralı"dır.

12. Kendini dene

Önce kâğıda çöz, sonra aşağıdaki cevaplarla karşılaştır.

1. "Çay demle" işi için şu adımları doğru sıraya koy: suyu demliğe dök, suyu kaynat, çayı demliğe koy, bekle.
2. `bool odevBitti = "false";` satırındaki hata nedir?
3. `Console.WriteLine(18 - 2 * 4);` ve `Console.WriteLine((18 - 2) * 4);` satırlarının çıktıları nedir?
4. `int.TryParse("12a", out x)` true mu false mu döner? x kaç olur?
5. Kural "hız en fazla 90 ise uyarı yok" ise doğru koşul hangisidir: `hiz < 90` mu, `hiz <= 90` mı?
6. `for (int i = 2; i <= 10; i += 4) Console.WriteLine(i);` ekrana ne yazar, gövde kaç kez çalışır?
7. `static int Ucle(int s) { return s * 3; }` için `Console.WriteLine(Ucle(Ucle(2)));` ne yazar?
8. Bir `Kutu` sınıfında `private int agirlik;` ve yalnız `get` içeren `public int Agirlik` özelliği var. `k.Agirlik = 7;` satırı neden hata verir?
9. Kalem kutusu en fazla 12 kalem alır. `Ekle()` metodu kalem sayısını 3 artırsın, 12'yi aşarsa 12 yapsın. Metodu yaz; 8, 10 ve 12 için sonucu bul.
10. `Al()` metodu: stok 5'ten azsa stok değişmeden false, aksi hâlde stok 5 azalsın ve true dönsün. Metodu yaz; 4 ve 5 için (son stok, dönüş) çiftlerini bul.

Cevaplar

1. Suyu kaynat → çayı demliğe koy → suyu demliğe dök → bekle. (Başa "Başla", sona "Bitir" eklenebilir.)
2. bool değer tırnaksız yazılır: `bool odevBitti = false;` Tırnaklı hâli metindir; bu bir derleme hatasıdır.
3. 10 ve 64. İlkinde önce çarpma yapılır, ikincisinde önce parantez.
4. false; x = 0.
5. `hiz <= 90`, çünkü "en fazla 90" 90'ı da kapsar.
6. 2, 6, 10 — gövde 3 kez çalışır.
7. Ucle(2) = 6, Ucle(6) = 18 → 18.
8. Özellikte `set` yok, yani salt okunurdur. Alan private olduğu için dışarıdan da değiştirilemez.
9. `kalem = kalem + 3; if (kalem > 12) kalem = 12;` → 8 için 11, 10 için 12, 12 için 12.

10. `if (stok < 5) return false; stok = stok - 5; return true;` → 4 için (4, false), 5 için (0, true).

Başarılar!